O Problema do Jantar dos Filósofos

Projeto de Sistemas Operacionais (T303) | Prof. MSc. Felipe Jucá dos Santos
Aluno: Alternandes de Castro | Matrícula: 2012554 | Código/Turma: T303-26 Proj de Sistema Operacional

1. Explicação da implementação

O programa foi escrito em Java, em um único arquivo, sem bibliotecas externas. Cada filósofo é uma thread da classe `Filosofo`, que repete dez vezes o ciclo pedido: `pensar()` aguarda um tempo aleatório de até 500 ms, `pegarGarfos()` bloqueia até obter os dois garfos, `comer()` aguarda um tempo aleatório de até 400 ms e `devolverGarfos()` libera os recursos.

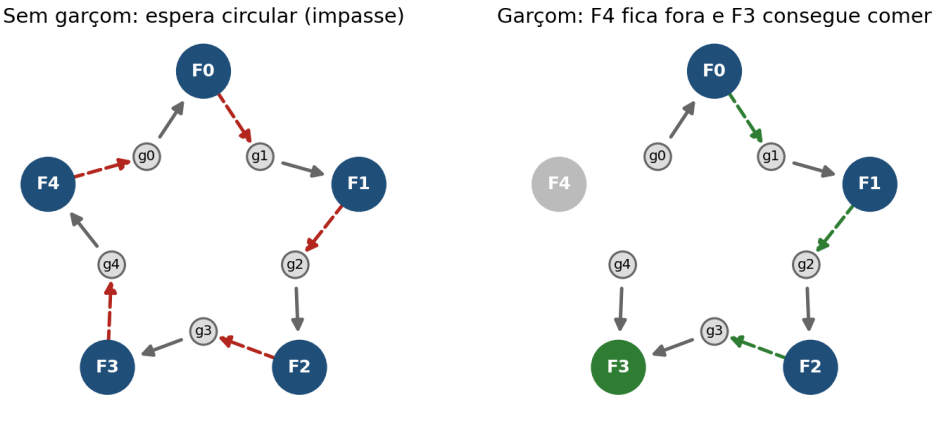
Cada garfo é um semáforo com uma permissão, portanto só um filósofo o segura por vez. O garfo i fica à esquerda do filósofo i, e o garfo (i+1) mod 5, à direita. O garçom é um semáforo com quatro permissões. A parte central da solução é:

```
garcom.acquire();                  // no máximo 4 filósofos disputam os garfos
garfos[garfoEsquerdo].acquire();   // bloqueia até o garfo esquerdo ficar livre
garfos[garfoDireito].acquire();    // com os dois garfos, pode comer
```

O log mostra os quatro eventos exigidos, por exemplo: "Filósofo 2 começa a pensar", "Filósofo 2 tenta pegar os garfos 2 e 3", "Filósofo 2 consegue comer" e "Filósofo 2 devolve os garfos 2 e 3". O programa só termina depois que todos os filósofos comem todas as refeições.

2. Análise dos problemas encontrados

Impasse (deadlock). Sem o garçom, todos pegam primeiro o garfo esquerdo. Se os cinco fizerem isso ao mesmo tempo, cada um fica com um garfo esperando o do vizinho, e ninguém avança. As quatro condições de Coffman estão presentes: exclusão mútua (o garfo não é compartilhável), posse e espera (segura um garfo enquanto espera o outro), não preempção (ninguém tira o garfo do vizinho) e espera circular (Figura 1). Basta eliminar uma delas para o impasse se tornar impossível.



Seta contínua: garfo na mão do filósofo. Seta tracejada: filósofo esperando o garfo.

Figura 1. A espera circular sem garçom e como o garçom a interrompe.

O impasse é intermitente. Sem garçom e sem pausa, as 20 execuções terminaram normalmente, embora o código esteja errado, porque o intervalo entre pegar os dois garfos é muito curto. Para reproduzir o erro foi preciso inserir uma pausa de 500 ms nesse intervalo: então as 20 execuções travaram, sem nenhuma refeição. Ou seja, um programa incorreto pode passar em testes comuns e falhar quando o escalonamento muda.

Inanição (starvation). Mesmo sem impasse, um filósofo pode esperar indefinidamente se os vizinhos se alternarem nos garfos e sempre passarem à sua frente. Um semáforo comum não garante a ordem de atendimento, por isso os semáforos foram criados no modo justo, que atende por ordem de chegada.

Ordem do log. Se a devolução fosse registrada depois de liberar os garfos, o log poderia mostrar o vizinho comendo antes de o garfo ser devolvido. Por isso o registro é feito antes da liberação.

3. Estratégia de solução escolhida

A solução escolhida é o **garçom com filas justas**. Antes de tocar nos garfos, o filósofo pede permissão a um semáforo que admite no máximo N-1 = 4 filósofos. Essa regra elimina a espera circular, pois o ciclo exige os cinco. Com quatro filósofos e cinco garfos, mesmo que cada um segure um garfo, sobra um garfo livre ao lado de algum deles; esse filósofo come e libera os recursos (Figura 1). Quanto à inanição, o garçom e os garfos atendem por ordem de chegada e o tempo de comer é finito, logo quem entra em uma fila é atendido depois de um número limitado de outros.

A Tabela 1 resume os testes. Nas execuções com garçom, a leitura dos logs não encontrou dois vizinhos comendo ao mesmo tempo.

Tabela 1. 20 execuções por configuração. Foi considerada travada a execução que não terminou em 60 s (90 s na última linha).

Configuração	Execuções concluídas	Refeições por filósofo
Sem garçom, com pausa de 500 ms	**0 de 20**	nenhuma (impasse)
Sem garçom, sem pausa	20 de 20	10
Com garçom, sem pausa	20 de 20	10
Com garçom, com pausa de 500 ms	20 de 20	10

A hierarquia de recursos (pegar sempre o garfo de menor número primeiro) também elimina a espera circular, sem precisar de um componente central, mas a ordem imposta pode favorecer alguns filósofos. O monitor de estados clássico foi descartado porque nele dois vizinhos que se alternam podem deixar o filósofo do meio sem comer. O garçom foi escolhido por ser simples e por tratar impasse e inanição juntos. Seu custo é um ponto central de coordenação e uma permissão a mais para pedir antes de comer.

4. Conclusões sobre sincronização em sistemas operacionais

Exclusão mútua não basta. A versão sem garçom protege cada garfo corretamente e, ainda assim, para por completo. Um programa concorrente precisa de segurança (nada de errado acontece) e também de vivacidade (o trabalho acaba sendo feito).

Teste não prova ausência de impasse. As 20 execuções sem falha da versão errada mostram isso. A confiança na solução vem de um argumento sobre a estrutura do programa, a eliminação de uma condição de Coffman; os testes apenas reforçam esse argumento. É por isso que condições de corrida são tão difíceis de depurar: uma pequena mudança de tempo faz o erro aparecer ou desaparecer.

Prevenir é uma decisão de projeto, e tem custo. Sistemas reais fazem escolhas semelhantes: o núcleo do Linux adota ordem de aquisição de locks e a verifica com a ferramenta lockdep (prevenção), enquanto bancos de dados costumam detectar o ciclo e abortar uma transação (detecção e recuperação). Semáforos contadores, como o garçom, são o mecanismo usual para limitar o acesso a um conjunto de recursos.

Referências

COFFMAN, E. G.; ELPHICK, M. J.; SHOSHANI, A. System Deadlocks. ACM Computing Surveys, v. 3, n. 2, p. 67-78, 1971.

DIJKSTRA, E. W. Hierarchical Ordering of Sequential Processes. Acta Informatica, v. 1, n. 2, p. 115-138, 1971.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Operating System Concepts. Wiley.

TANENBAUM, A. S.; BOS, H. Sistemas Operacionais Modernos. 4. ed. Pearson, 2016.